

Détection de fraude par carte bancaire à l'aide de l'apprentissage automatique

Chaymae Al-aissaoui

Département d'informatique - Faculté polydisciplinaire de Ouarzazate, Université Ibn Zohr, Ouarzazate, Maroc

Email : aissachaimae@gmail.com

Abstract—Cette étude présente une démarche complète de détection de fraude par carte bancaire à l'aide de Machine Learning appliquée au dataset : Credit Card Fraud Detection. Après la phase d'exploration et de preprocessing, les outliers ont été traités par IQR capping afin de limiter l'influence des extrêmes sans supprimer les observations, puis les variables ont été standardisées et les données séparées en ensembles d'entraînement et de test. Le Dataset, déjà transformé par ACP (Analyse en Composantes Principales) pour la majorité des variables (V1–V28) qui permis de travailler sur une représentation compacte tout en conservant les informations essentielles. Quatre modèles de classification ont été entraînés et optimisés par Grid Search : Arbre de décision, Forêt aléatoire, SVM (noyau RBF) et XGBoost. Les performances ont été comparées à l'aide des métriques adaptées au fort déséquilibre des classes, notamment Precision, Recall, F1-score, ROC-AUC, PR-AUC, ainsi que Balanced Accuracy et MCC, complétées par les matrices de confusion. Les résultats montrent l'importance d'une évaluation centrée sur les faux négatifs et la capacité de généralisation, afin d'améliorer la détection des transactions frauduleuses dans un contexte réaliste.

Index Terms—Fraud detection, Imbalanced classification, ACP, Outlier capping (IQR), Grid Search, SVM, Random Forest, Decision Tree, XGBoost, ROC-AUC, PR-AUC, MCC.

I. INTRODUCTION

Alors que les paiements électroniques se généralisent et que les services financiers se digitalisent, la fraude par carte bancaire demeure une menace majeure pour les banques, les commerçants et les utilisateurs. Les fraudeurs profitent de la rapidité des transactions et du grand nombre d'opérations effectuées chaque jour pour cacher leurs activités illégales parmi des transactions normales. Dans ce contexte, il devient difficile de repérer ces fraudes manuellement, ce qui rend la détection automatique des transactions frauduleuses particulièrement important.

L'un des principaux défis de la détection de fraude est le fort déséquilibre entre les classes. Dans le Dataset Credit Card Fraud Detection de Kaggle, utilisé dans ce travail, on trouve 284 807 transactions, dont seulement 492 sont frauduleuses, soit environ 0,17% du total. Dans ces conditions, un modèle simple qui prédit toujours la classe majoritaire (classe 0) peut obtenir une très bonne accuracy, sans être utile en pratique, car il manquerait la majorité des fraudes. Il est donc essentiel de concevoir et d'évaluer les modèles en tenant compte de ce déséquilibre, notamment en considérant le coût élevé des faux négatifs (fraudes non détectées), tout en limitant les faux positifs (transactions légitimes rejetées).

La qualité et la cohérence des données d'entraînement jouent un rôle essentiel dans la performance des modèles d'apprentissage automatique. Cela passe par plusieurs étapes de prétraitement, notamment la vérification des valeurs manquantes, la sélection des variables pertinentes, la standardisation des variables numériques et le traitement des valeurs aberrantes. Dans ce travail, notre dataset ne contient pas de valeurs manquantes, ce qui permet d'éviter l'étape d'imputation. Aussi, afin de réduire l'impact des valeurs extrêmes, un traitement des outliers basé sur la méthode IQR (capping) a été appliqué, sans supprimer de lignes. Par ailleurs, les variables V1 à V28 viennent d'une Analyse en Composantes Principales (ACP), ce qui permet à la fois de préserver la confidentialité des données et d'obtenir une représentation plus compacte des transactions. Pour nos expérimentations, nous avons sélectionné un sous-ensemble de variables (Time, Amount, V1, V2, V3 et Class), aboutissant à un jeu de données final de dimension (284 807, 6), plus simple à exploiter et à interpréter.

Dans ce cadre, nous menons une étude comparative de plusieurs modèles de classification supervisée couramment utilisés en détection de fraude, à savoir : Arbre de décision, Forêt aléatoire, SVM (noyau RBF) et XGBoost. Les hyperparamètres ont été optimisés à l'aide de Grid Search et les modèles ont été évalués via des métriques adaptées aux données déséquilibrées : Precision, Recall, F1-score, ROC-AUC, PR-AUC, ainsi que des indicateurs avancés tels que Balanced Accuracy et le Matthews Correlation Coefficient (MCC), complétés par l'analyse des matrices de confusion. Les résultats obtenus montrent notamment des compromis importants entre détection des fraudes (Recall) et maîtrise des fausses alertes (Precision) : le SVM présente un recall élevé au prix d'un nombre important de faux positifs, tandis que la Forêt aléatoire et XGBoost offrent une meilleure précision mais un recall plus limité. Ces observations soulignent la nécessité d'une sélection de modèle guidée par l'objectif métier (réduire les fraudes manquées vs limiter les alertes inutiles) plutôt que par l'accuracy seule. La suite du travail présente les données, la méthodologie, puis l'analyse des résultats.

II. ÉTAT DE L'ART

Dans cette étude, nous comparons quatre approches d'apprentissage supervisé largement utilisées pour la

classification binaire et fréquemment rencontrées dans la détection des fraudes bancaires : l'Arbre de décision, la Forêt aléatoire, la Machine à vecteurs de support (SVM) et XGBoost. Ces méthodes représentent des familles de modèles complémentaires, ce qui permet d'analyser la performance et la complexité.

A. Présentation des algorithmes utilisés

1) *Arbre de décision (Decision Tree)* : Un arbre de décision prend des décisions basées sur des tests effectués sur les caractéristiques des données. À chaque nœud, une condition est appliquée pour diviser les données, jusqu'à ce qu'une décision soit prise au niveau des feuilles de l'arbre. Le modèle est particulièrement apprécié pour sa simplicité et son interprétabilité, car il est possible de suivre le chemin de décision expliquant une prédiction.

2) *Forêt aléatoire (Random Forest)* : Le random forest est un algorithme de machine learning couramment utilisé, qui combine les résultats de plusieurs arbres de décision pour obtenir un résultat unique. La prédiction finale est obtenue par vote majoritaire (classification). Cette stratégie améliore la généralisation, ce qui explique son usage fréquent dans des tâches de détection d'anomalies et de fraude.

3) *SVM avec noyau RBF* : SVM est un algorithme de classification qui cherche à trouver la frontière de décision optimale séparant les différentes classes avec une marge maximale. Cette frontière est définie par un sous-ensemble de points appelés vecteurs de support. Lorsque les données ne sont pas séparables linéairement, l'utilisation de noyau permet de projeter les données dans un espace de dimension plus élevée. Le noyau RBF (Radial Basis Function) permet de modéliser des frontières de décision non linéaires et complexes, ce qui est nécessaire pour distinguer des schémas de fraude.

4) *XGBoost (Extreme Gradient Boosting)* : XGBoost est une bibliothèque de ML qui utilise des arbres de décision auxquels est appliqué le boosting de gradient, ce dernier étant un algorithme de boosting d'apprentissage supervisé qui utilise la descente de gradient. C'est à dire chaque nouvel arbre est entraîné pour corriger les erreurs des arbres précédents selon une optimisation par gradient. Il est connu par sa vitesse, son efficacité et sa capacité à s'adapter à de grands Datasets.

B. Avantages et inconvénients

Les modèles étudiés présentent plusieurs avantages pour la détection de fraude. Ils permettent de traiter bien les données tabulaires et de modéliser des relations potentiellement non linéaires entre variables, ce qui est essentiel pour distinguer des comportements frauduleux souvent subtils. L'arbre de décision fournit une interprétabilité via des règles explicites,

tandis que la forêt aléatoire améliore la robustesse et la capacité de généralisation en combinant plusieurs arbres. Le SVM-RBF est adapté aux séparations. Enfin, XGBoost est reconnu pour ses performances élevées grâce à ces mécanismes de régularisation, ce qui en fait une bonne approche dans les application de classification.

Même si tout ça, l'utilisation de ces modèles avoir aussi des limites, notamment dans le cas de déséquilibre extrême des classes comme la fraude bancaire. Un arbre de décision peut facilement sur-apprendre si sa complexité n'est pas contrôlée, La forêt aléatoire et XGBoost, sont moins interprétables et nécessitent un réglage des hyperparamètres pour éviter le surapprentissage et pour gérer le déséquilibre. Le SVM-RBF est sensible au choix de ses hyperparamètres, peut être très coûteux en calcul sur de grands ensembles, et reste difficile à interpréter. Globalement, ces modèles peuvent donner une accuracy élevée en manquant des fraudes si l'évaluation n'est pas basée sur des métriques adaptées.

C. Les travaux portant sur des problématiques similaires

Plusieurs travaux ont étudié la détection de fraude par carte bancaire à l'aide de techniques d'apprentissage automatique, en particulier sur des données fortement déséquilibrées. Par exemple, une étude comparative sur des modèles supervisés met en avant l'utilisation de SVM, Decision Tree et Random Forest pour la détection de fraude et discute leurs performances sur des ensembles de transactions bancaires, en soulignant le rôle des métriques autres que l'accuracy dans ce contexte[1]. D'autres comme une étude de synthèse publiée sur arXiv, présentent des méthodes de machine learning appliquées à la fraude et insistent sur l'importance d'évaluer les modèles avec ROC-AUC et surtout PR-AUC lorsque la classe positive est rare[3]. Enfin une vue de la littérature sur la fraude « cyber » liée aux cartes bancaires analyse les algorithmes les plus utilisés, les stratégies de traitement du déséquilibre et les métriques adaptées, montrant que la comparaison sur des données déséquilibrées nécessite des indicateurs comme F1-score et MCC [2].

III. MÉTHODOLOGIE

A. Dataset Description

Nous utilisons Dataset public Credit Card Fraud Detection (Kaggle), contenant 284,807 transactions effectuées par carte bancaire. La variable cible *Class* indique si une transaction est frauduleuse (*Class*=1) sinon (*Class*=0). Le dataset original comporte 31 variables : *Time*, *Amount* et 28 variables anonymisées (*V1*–*V28*) obtenues par ACP.

Dans nos expériences, afin de simplifier l'analyse, nous retenons un sous-ensemble de caractéristiques composé de *Time*, *Amount*, *V1*, *V2* et *V3*, en plus de la variable cible *Class*. La distribution des classes est fortement déséquilibrée, avec 492 transactions frauduleuses (0,17%) contre 284,315 transactions légitimes (99,83%).

TABLE I
DESCRIPTION DU DATASET UTILISÉ

Élément	Description
Nom du dataset	Credit Card Fraud Detection (Kaggle)
Nombre d'instances	284 807 transactions
Variables (originales)	31 (Time, Amount, V1–V28, Class)
Nature des variables V1–V28	Issues d'une ACP (données anonymisées)
Variable cible	Class (0 : légitime, 1 : fraude)
Répartition des classes	Classe 0 : 284 315 (99.83%) Classe 1 : 492 (0.17%)
Variables retenues	Time, Amount, V1, V2, V3, Class
Dimension finale	(284 807 × 6)

La matrice de corrélation montre les variables sélectionnées. Chaque cellule est coloré selon la valeur de corrélation : une valeur proche de +1 indique une relation linéaire positive forte, une valeur proche de -1 une relation négative forte, et une valeur proche de 0 signifie qu'il existe peu ou pas de relation linéaire.

Dans notre cas, la plupart des corrélations sont faibles, ce qui signifie une dépendance linéaire limitée entre les variables. On remarque une corrélation négative plus visible entre Amount et V2 ($\approx -0,53$) et entre Time et V3 ($\approx -0,42$). Enfin, la corrélation avec la cible Class reste globalement faible, ce qui indique que la fraude ne peut pas être expliquée par une seule variable et nécessite une approche de classification spéciale.

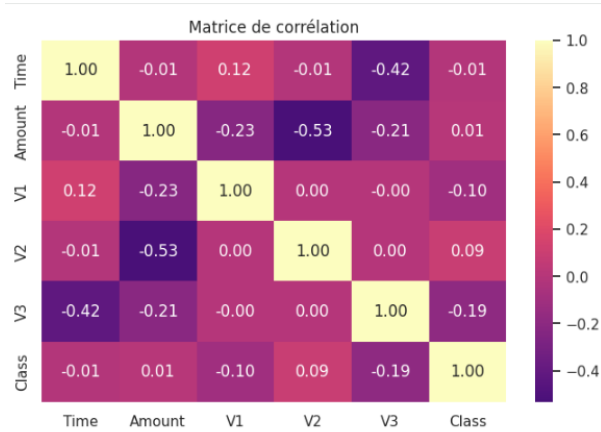


Fig. 1. Matrice de corrélation du dataset creditcard

B. Prétraitement

Le prétraitement a été réalisé en plusieurs étapes. Tout d'abord, une vérification des valeurs manquantes a confirmé l'absence de données manquantes, ce qui a rendu inutile toute imputation. Ensuite, afin de simplifier l'expérimentation, une sélection de variables a été effectuée en conservant uniquement Time, Amount, V1, V2, V3 et Class, ce qui conduit à un jeu final de dimension (284 807, 6). Les valeurs aberrantes ont été traitées à l'aide de la technique IQR capping, qui consiste à plafonner les valeurs extrêmes sans supprimer de lignes. Enfin, une standardisation a été appliquée pour mettre les variables

sur un échelle comparable, en particulier pour les modèles sensibles à l'échelle comme le SVM.

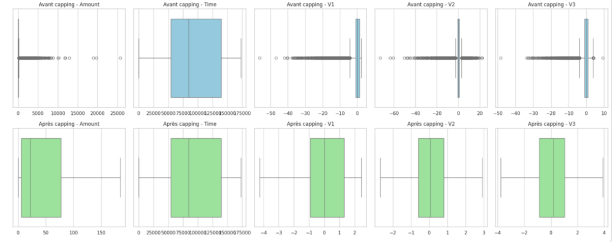


Fig. 2. Avant et après traitement des outliers

La Fig. 2 présente des boxplots des variables sélectionnées avant et après le traitement des valeurs aberrantes par IQR capping. Avant capping, plusieurs variables présentent des valeurs extrêmes. Après capping, ces valeurs sont plafonnées aux bornes définies par l'IQR, ce qui réduit l'impact des extrêmes tout ça avec aucune suppression de lignes. Cette étape vise à stabiliser l'entraînement des modèles et à améliorer leur robustesse.

C. Configuration expérimentale

Les données ont été séparées en ensemble d'entraînement et ensemble de test selon le ratio 80/20, en utilisant une division stratifiée (stratify = y) afin de préserver la proportion très faible de la classe minoritaire (fraude) dans les deux ensembles. Les hyperparamètres des modèles ont été optimisés à l'aide de GridSearch sur l'ensemble d'entraînement. A cause du fort déséquilibre des classes, le critère d'optimisation retenu est l'Average Precision, plus informatif que l'accuracy dans ce contexte. La validation croisée utilisée durant la recherche d'hyperparamètres est fixée à cv = 3 pour l'arbre de décision et la forêt aléatoire, et à cv = 2 pour le SVM et XGBoost. Une fois les meilleurs hyperparamètres sélectionnés, l'évaluation finale est réalisée sur l'ensemble de test à l'aide des métriques adaptées.

D. Choix des modèles

Afin d'obtenir une comparaison représentative et cohérente, on a utilisé quatre modèles supervisés.

- Arbre de décision : est utilisé comme modèle de référence, car il est simple à entraîner, rapide et relativement interprétable ce qui facilite l'analyse des décisions dans un contexte sensible comme la fraude.

TABLE II
HYPERPARAMÈTRES D'ARBRE DE DÉCISION (GRID SEARCH, SCORING = AVERAGE PRECISION)

Hyperparamètre	Valeur
max_depth	None
min_samples_split	10

- Forêt aléatoire : a été sélectionnée pour sa robustesse en combinant plusieurs arbres entraînés sur des sous-échantillons aléatoires, elle réduit la variance d'un arbre

unique et améliore généralement la généralisation, ce qui est particulièrement utile lorsque les données sont bruitées.

TABLE III
HYPERPARAMÈTRES DE FORÊT ALÉATOIRE (GRID SEARCH, SCORING = AVERAGE PRECISION)

Hyperparamètre	Valeur
n_estimators	200
max_depth	None
min_samples_split	2

- SVM avec noyau RBF : a été choisi pour sa capacité à apprendre des frontières de décision non linéaires et complexes.

TABLE IV
HYPERPARAMÈTRES DE SVM (RBF) (GRID SEARCH, SCORING = AVERAGE PRECISION)

Hyperparamètre	Valeur
kernel	rbf
C	10
gamma	scale

- XGBoost : a été retenu car il s'agit d'un algorithme de boosting reconnu pour ses performances élevées sur les données tabulaires, il permet de capturer des interactions non linéaires tout en intégrant des mécanismes de régularisation, et il est performant dans les applications de détection de fraude.

TABLE V
HYPERPARAMÈTRES DE XGBOOST (GRID SEARCH, SCORING = AVERAGE PRECISION)

Hyperparamètre	Valeur
n_estimators	100
max_depth	6
learning_rate	0.1

E. Métriques d'évaluation

Dans ce problème de détection de fraude, la classe positive (fraude) est très rare, ce qui rend l'accuracy insuffisante à elle seule. Nous évaluons donc les modèles à l'aide des métriques suivantes, calculées à partir de la matrice de confusion : vrais positifs (TP), faux positifs (FP), vrais négatifs (TN) et faux négatifs (FN).

1) *Accuracy*: L'accuracy mesure la proportion totale de prédictions correctes :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Dans le cas de données très déséquilibrées, une valeur élevée peut être obtenue en prédisant majoritairement la classe négative.

2) *Précision (Precision)*: La précision mesure la proportion de transactions prédites frauduleuses qui sont réellement frauduleuses :

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (2)$$

Une précision élevée indique peu de fausses alertes (faibles FP).

3) *Rappel (Recall)*: Le rappel mesure la capacité à détecter les fraudes :

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (3)$$

Un rappel élevé est important pour réduire les fraudes non détectées (faibles FN).

4) *F1-score*: Le F1-score est la moyenne harmonique entre précision et rappel :

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (4)$$

Il permet de résumer le compromis entre détection (Recall) et contrôle des fausses alertes (Precision).

5) *Balanced Accuracy*: La Balanced Accuracy tient compte de l'importance des deux classes en moyennant la sensibilité et la spécificité :

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right). \quad (5)$$

6) *ROC-AUC*: La courbe ROC représente le compromis entre le taux de vrais positifs (TPR) et le taux de faux positifs (FPR) pour différents seuils :

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (6)$$

Le score ROC-AUC mesure la capacité du modèle à classer un exemple positif au-dessus d'un exemple négatif. Mais sur des données très déséquilibrées, ROC-AUC peut rester élevé même avec une précision faible.

7) *PR-AUC (Average Precision)*: Cette courbe (PR) est souvent plus informative lorsque la classe positive est rare. L'Average Precision (AP) correspond à la courbe PR et peut être approximée par :

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n, \quad (7)$$

où P_n et R_n représentent respectivement la précision et le rappel aux différents seuils. Une valeur élevée de PR-AUC indique une meilleure performance sur la classe positive.

8) *Matthews Correlation Coefficient (MCC)*: Le MCC est une mesure corrélationnelle robuste pour les classes déséquilibrées :

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (8)$$

Le MCC varie entre -1 (mauvaise classification), 0 (aléatoire) et 1 (parfait).

Ce schéma résume le pipeline global suivi dans la méthodologie, depuis le prétraitement des données jusqu'à l'évaluation finale et la comparaison des modèles.

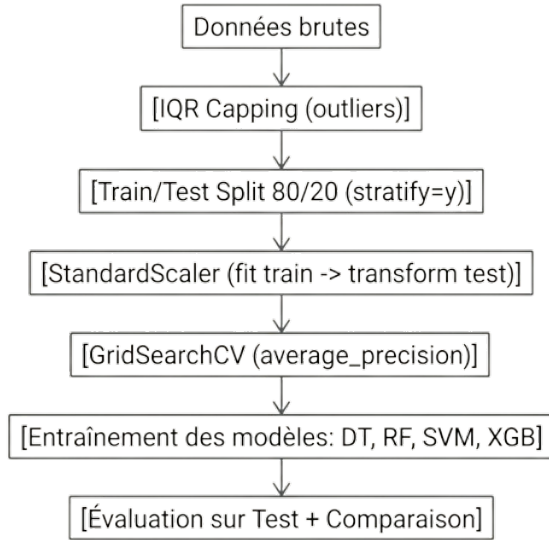


Fig. 3. Shéma de la méthodologie

IV. APPLICATION

A. Implémentation

L'implémentation a été réalisée sous Google Colab (Python ≈ 3.11) en suivant un flux de traitement séquentiel (sans utilisation de pipeline). Après le chargement du dataset, un traitement des valeurs aberrantes a été appliqué à l'aide de la méthode IQR capping.

Pour l'ensemble des variables numériques, à l'exception de la variable cible *Class*, les premier et troisième quartiles ($Q1$ et $Q3$) sont calculés, puis l'intervalle IQR a été déterminé comme suit : $IQR = Q3 - Q1$. Les valeurs situés en dehors de l'intervalle $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ ont été ramenées aux bornes correspondantes à l'aide d'une opération de *clipping*, sans suppression de lignes.

Ensuit, Les données ont été divisées en ensembles d'entraînement et de test selon un ratio 80/20, avec une séparation stratifiée ($stratify=y$) et un $random_state=42$ afin de garantir la reproductibilité. La standardisation a été réalisée à l'aide de *StandardScaler*, en ajustant les paramètres uniquement sur l'ensemble d'entraînement ($fit_transform$), puis en appliquant la transformation à l'ensemble de test ($transform$) afin d'éviter toute fuite d'information (*data leakage*). Les données standardisées ont été reconverties en *DataFrame* pour conserver les noms des variables.

L'optimisation des hyperparamètres a été effectuée à l'aide de *GridSearch*, en utilisant $average_precision$ (PR-AUC) comme fonction de score et $n_jobs=-1$ pour paralléliser les calculs. La validation croisée a été fixée à $cv=3$ pour l'arbre de décision et la forêt aléatoire, et à $cv=2$ pour le SVM et XGBoost. Les prédictions finales, utilisées pour la

matrice de confusion et les métriques de classification, ont été obtenues avec la méthode `predict()`.

B. Outils utilisés

Les expérimentations ont été menées en Python en s'appuyant sur les bibliothèques suivantes : NumPy et Pandas pour la manipulation des données, scikit-learn pour la séparation des données, la standardisation, l'entraînement des modèles (Arbre de décision, Forêt aléatoire, SVM), l'optimisation par Grid Search ainsi que le calcul des métriques. Le modèle XGBoost a été entraîné à l'aide de la bibliothèque xgboost. Les visualisations ont été générées avec Matplotlib et Seaborn.

V. RÉSULTATS ET DISCUSSION

A. Tableaux comparatifs

TABLE VI
PERFORMANCES SUR L'ENSEMBLE DE TEST (DECISION TREE ET RANDOM FOREST)

Métrique	Decision Tree	Random Forest
Accuracy	0.9974	0.9989
Precision	0.3311	0.8367
Recall	0.5000	0.4184
F1-score	0.3984	0.5578
ROC-AUC	0.7494	0.9111
PR-AUC	0.3021	0.5590
Balanced Acc.	0.7491	0.7091
MCC	0.4056	0.5912

TABLE VII
PERFORMANCES SUR L'ENSEMBLE DE TEST (SVM ET XGBOOST)

Métrique	SVM (RBF)	XGBoost
Accuracy	0.9555	0.9987
Precision	0.0286	0.8286
Recall	0.7551	0.2959
F1-score	0.0552	0.4361
ROC-AUC	0.8696	0.9627
PR-AUC	0.0448	0.4701
Balanced Acc.	0.8555	0.6479
MCC	0.1416	0.4947

La Table VIII compare le F1-score sur train et sur test. La Random Forest et l'arbre de décision ont un grand écart : ils sont très bons sur train mais moins bons sur test, donc ils font du surapprentissage. Le SVM a presque le même F1 sur train et test, mais les deux sont faibles : il fait plutôt du sous-apprentissage. XGBoost a un écart plus petit, donc il généralise mieux que les deux premiers.

TABLE VIII
VÉRIFICATION DU SURAPPRENTISSAGE/SOUS-APPRENTISSAGE VIA
L'ÉCART DE F1-SCORE ENTRE TRAIN ET TEST.

Modèle	F1 (train)	F1 (test)	Gap
Decision Tree	0.7665	0.3984	0.3682
Random Forest	1.0000	0.5578	0.4422
SVM (RBF)	0.0738	0.0552	0.0186
XGBoost	0.5441	0.4361	0.1081

B. Analyse des résultats

1) Observations générales:

- Les accuracies sont très élevées pour Decision Tree, Random Forest et XGBoost ($\approx 0.997 - 0.999$), ce qui était attendu vu le déséquilibre des classes. Il ne suffit donc pas pour juger la qualité de détection.
- Les métriques orientées classe 1 montrent des différences nettes : PR-AUC et MCC distinguent mieux les modèles.

2) Random Forest : meilleur compromis global dans les résultats:

- Meilleure PR-AUC = 0.5590 et meilleur MCC = 0.5912.
- Très bonne précision = 0.8367 (peu de fausses alertes) et rappel modéré (0.4184).
- En matrice de confusion : FP=8 (très faible) mais FN=57 (fraudes ratées encore importantes).

3) XGBoost : excellent classement (ROC-AUC), mais rappel plus faible:

- ROC-AUC le plus élevé (0.9627), ce qui indique une très bonne capacité de séparation.
- Mais le rappel est faible (0.2959) : il rate plus de fraudes (FN=69) que Random Forest.
- FP=6 (très faible) et précision élevée (0.8286) : bon pour limiter les alertes inutiles.

4) Decision Tree : baseline interprétable mais moins performant:

- Rappel 0.5 (mieux que RF et XGB) mais précision faible (0.331), donc plus de fausses alertes (FP=99).
- PR-AUC et MCC restent nettement inférieurs à Random Forest.
- Utile comme modèle explicatif/baseline, mais moins adapté pour un compromis robuste.

5) SVM : rappel élevé mais trop de faux positifs:

- Rappel 0.7551 (le meilleur) mais précision très faible (0.0286)
- FP=2509 (énorme) pour seulement TP=74.
- PR-AUC extrêmement faible (0.0448) et MCC faible.

6) Conclusion discussion:

- Pour le cas d'un compromis entre détecter des fraudes et limiter les fausses alertes, Random Forest est le meilleur dans tes résultats (PR-AUC et MCC les plus élevés).
- XGBoost est prometteur (ROC-AUC élevé, FP très faible), mais semble nécessiter ajustement de seuil ou gestion du déséquilibre pour améliorer le rappel.
- Pour le cas de minimiser les fraudes (réduire FN) : SVM obtient le meilleur rappel et détecte le plus de fraudes, mais au prix d'un très grand nombre de fausses alertes.

- Pour le cas de minimiser les fausses alertes (réduire FP) : XGBoost et Random Forest sont les plus adaptés car ils génèrent très peu de faux positifs, tout en conservant une précision élevée.

C. Graphiques

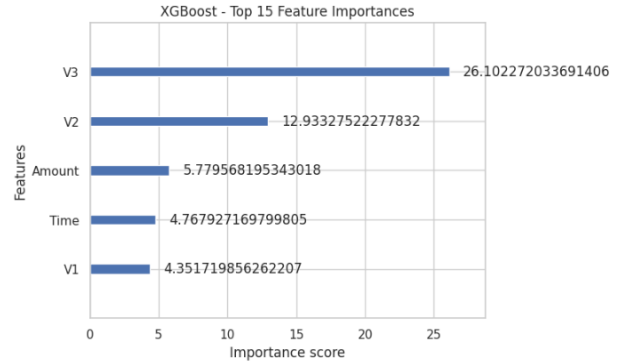


Fig. 4. Importance des variables selon XGBoost

La Fig 3 montre les variables les plus importantes selon le modèle XGBoost. On voit que V3 est la plus influente, suivie de V2. Les variables Amount, Time et V1 ont un impact plus faible sur les décisions du modèle.

VI. CONCLUSION

A. Résumé

Dans ce travail, nous avons appliqué plusieurs modèles de classification pour la détection de fraude sur un Dataset fortement déséquilibré. L'évaluation a été réalisée avec des métriques adaptées. Globalement, la Random Forest a donné le meilleur compromis entre la détection des fraudes et la limitation des fausses alertes, tandis que XGBoost a montré un bon potentiel mais avec un rappel plus faible dans notre configuration.

B. Perspectives

Pour améliorer les résultats, il serait bien de tester des techniques de rééquilibrage. Une validation croisée plus robuste et une analyse plus approfondie des variables importantes pourraient également renforcer la fiabilité et l'interprétation du modèle.

VII. REFERENCES

- [1] Credit Card Fraud Detection Using SVM, Decision Tree and Random Forest, Available : https://link.springer.com/chapter/10.1007/978-3-031-28387-1_27
- [2] A systematic review of literature on credit card cyber fraud detection using machine and deep learning, PubMed Central (PMC). Available : <https://pmc.ncbi.nlm.nih.gov/articles/PMC10280638/>
- [3] Credit Card Fraud Detection using Machine Learning: A Study, arXiv. Available : <https://arxiv.org/abs/2108.10005>

Kamal. Omari, Machine Learning — Chapitre 3 : Apprentissage supervisé (SVM, arbres de décision et métriques d'évaluation), support de cours, Faculté Polydisciplinaire de Ouarzazate, Université Ibn Zohr, Ouarzazate, Maroc, 2026

IBM, Random Forest, IBM Think, Available: <https://www.ibm.com/fr-fr/think/topics/random-forest>

IBM, XGBoost, IBM Think. Available: <https://www.ibm.com/fr-fr/think/topics/xgboost>

J. Davis and M. Goadrich, The Relationship Between Precision-Recall and ROC Curves, in Proc. ICML, 2006

F. Pedregosa et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011